LLM → Coding agent → Arnés → Loop → Orquestador → MCP

DEL SPEC AL HARNESS · ESTACIÓN 6

¿Tu spec está lista para implementar?

Hoy evaluamos la spec, la convertimos en un grafo de tareas y la dejamos en Linear, lista para que un agente la ejecute en la estación 7.

Salida de hoy: spec con nota A o B · design.md · paquete de tareas · grafo publicado o listo para publicar.



QUIÉN LES HABLA

Diógenes Moreira

CTO de 30X

5 años en Frubana, de VP of Engineering a CTO

El equipo de tecnología pasó de 10 a casi 100 personas, organizado por verticales.

8 años de tecnología en el sector público

Ministerio Público Fiscal y fundador de la Unidad de Ciberdelincuencia (UFECI).

Fractional CTO y advisor

rintin, Masu, ONA Report y Atina.app. Docente de posgrado en la UBA.

Autor

Arquitectura para Vibe-Coders, Vibe Coding para Desarrolladores y El martes que no estás.

No soy Leonardo González: hoy doy esta estación en su lugar. OpenSymphony y sus skills de planificación son trabajo suyo.

EL MAPA

Seis piezas, una cadena

Pieza	Qué hace	Dónde la usamos
LLM	Interpreta y propone. No tiene manos.	Detrás de todo
Coding agent	Un LLM con herramientas: lee, edita, ejecuta.	Claude Code, Codex, OpenHands
Arnés	Contexto, permisos, skills y hooks alrededor del agente.	Módulos 2, 3 y 4
Loop	Planear, editar, probar y corregir hasta un criterio.	Módulo 5 y E7
Orquestador	Toma tareas listas, respeta dependencias, junta evidencia.	Módulo 5 y E7
MCP	Conecta el agente con sistemas externos como Linear o GitHub.	Módulos 3 y 4

La barra de arriba te dice en qué pieza estamos en cada momento.

LLM → Coding agent → Arnés → Loop → Orquestador → MCP

CINCO MÓDULOS · CADA UNO ENTRA CON UN ARTEFACTO Y SALE CON OTRO

La ruta de hoy

Módulo	Min	Entra	Sale
1 · Del LLM al orquestador	17	Tu herramienta actual	Tu herramienta ubicada en el mapa
2 · Evaluar la spec	25	Tu spec de E4 y E5	spec-review.md con nota A–D
3 · design.md + Linear	17	Spec con nota A o B	design.md y fuentes de verdad claras
4 · Del plan al grafo	30	Spec + design.md	Paquete de tareas y grafo en Linear
5 · Loop y orquestador	12	Grafo publicado	Qué automatiza E7

Preguntas fuera del mapa van al estacionamiento: las respondemos al cierre.

MÓDULO 1 · DEL LLM AL ORQUESTADOR

El modelo propone. El agente actúa.

LLM

Predice la siguiente respuesta

Recibe texto y devuelve texto. No ve tu repo ni puede ejecutar nada por su cuenta.

Sirve para razonar, redactar y decidir el próximo paso.

Agente

Un LLM con herramientas

Puede leer archivos, editar, correr comandos y ver el resultado.

Claude Code, Codex u OpenHands son coding agents. Casi cualquier modelo entra en casi cualquier arnés: revisa qué permite tu suscripción.

El mismo modelo rinde distinto según lo que le damos alrededor.

MÓDULO 1 · DEL LLM AL ORQUESTADOR

Lo que rodea al agente

Arnés

Contexto, permisos y hábitos

AGENTS.md y design.md dan contexto. Las skills enseñan procedimientos. Los hooks reaccionan a eventos. Los permisos limitan qué puede tocar.

Ingeniería de contexto: todo lo necesario y solo lo necesario.

Loop

Intentar hasta un criterio

Planear → editar → probar → corregir. Termina cuando se cumple la aceptación, no cuando el agente deja de escribir.

Sin criterio de aceptación, el loop no sabe cuándo parar. Por eso hoy trabajamos la spec.

MÓDULO 1 · DEL LLM AL ORQUESTADOR

Anatomía de un arnés



MÓDULO 1 · DEL LLM AL ORQUESTADOR

Coordinar y conectar

Orquestador

Muchas tareas, en orden

Lee una cola de tareas, despacha las que no tienen bloqueos y junta la evidencia de cada una. Lo vemos con OpenSymphony en E7.

MCP

Un enchufe estándar

Model Context Protocol: la forma en que el agente habla con Linear, GitHub o tu documentación sin integraciones a medida.

Tu turno · 5 min: ubica tu herramienta en el mapa. ¿Qué piezas ya usas y cuál te falta?

MÓDULO 2 · EVALUAR LA SPEC

¿Le darías esta tarea a un agente?

INTENCIÓN AMBIGUA

“Implementa autenticación.”

¿Qué parte?
¿Con qué reglas?
¿Cómo sabemos que terminó?

TRABAJO PREPARADO

“Valida la política de contraseña.”

Reglas BR-PASS-01 a 04.
Función y pruebas; sin base de datos.
Casos normales y de error explícitos.

La diferencia no está en el código. Está en la spec.

MÓDULO 2 · SKILL EVALUATE-SPEC

Tres lentes para revisar una spec

1

Producto

Problema, usuario, alcance, fuera de alcance y reglas de negocio sin contradicciones.

2

TDD

Cada requisito tiene un resultado observable y casos de prueba, incluidos errores y bordes.

3

Diseño

Arquitectura mínima, contratos entre componentes y design.md para la interfaz.

La skill no reescribe tu spec: cita la evidencia y propone la corrección mínima.

MÓDULO 2 · NOTA ABCD

La nota define el siguiente paso

A

Lista

Todos los criterios críticos cumplen. Pasa al plan.

B

Lista con supuestos

Ningún crítico falta. Los supuestos quedan escritos y aceptados.

C

Necesita decisiones

Falta al menos un crítico. Resolver antes de planificar.

D

Todavía no es spec

Sin alcance o sin requisitos verificables. Volver a E4 y E5.

Una nota por ola, no por proyecto. Solo A o B pasan al módulo 4.

TU TURNO · 17 MIN

Evalúa tu spec

03 min	Abre tu repo y reúne la spec de E4 y E5.
06 min	Pide al arnés que lea y siga `skills/evaluate-spec/SKILL.md`.
05 min	Corrige el vacío más grave de `docs/spec-review.md`.
03 min	Vuelve a evaluar. Anota la nota antes y después.

Sin spec propia: usa EntreVista AI de la estación 5.

PAUSA · 5 MIN

Volvemos en cinco minutos

Al volver: ¿qué nota sacó tu spec?

MÓDULO 3 · DESIGN.MD + LINEAR

Una fuente de verdad para cada cosa

Artefacto	Qué gobierna	Si cambia
Spec en Git	Qué debe hacer el producto y sus reglas	Re-evaluar y ajustar las tareas afectadas
design.md	Cómo se ve y cómo se construye	Las tareas lo enlazan, no lo copian
Archivos de tarea	El contrato de cada tarea	Actualizar el archivo y su issue
Linear	Estado, dependencias y responsable	Se mueve con el trabajo
PR y tests	La evidencia de que se cumplió	Nueva revisión

No hay sincronización automática: si cambia el contrato, se actualizan archivo e issue.

MÓDULO 3 · DESIGN.MD

design.md: lo que el agente no debe adivinar

Qué contiene

Principios de experiencia
Sistema visual y componentes
Estados: carga, vacío, error, éxito
Arquitectura y contratos
Decisiones y qué no hacer

Cómo se usa

Vive en la raíz del repo.
El agente lo lee antes de cada tarea de interfaz o de contratos.
La skill de evaluación lo revisa en la lente de diseño.

Plantilla: estacion6/plantillas/design.md

Si una decisión de diseño está en el chat y no en design.md, el próximo agente no la conoce.

MÓDULO 3 · LINEAR VÍA MCP

Linear como cola de trabajo

Milestone
Agrupa tareas de un mismo objetivo.

→

Issue
Una tarea con su contrato y criterio de aceptación.

→

blockedBy
Las dependencias que forman el grafo.

→

area:
Etiquetas que agrupan por componente.

Con el MCP de Linear, el agente lee y actualiza issues desde el arnés. La interfaz web también sirve. ¿Usas Jira o ClickUp? Adapta la skill a tu tracker.

Publica en un proyecto sin orquestador observando: publicar puede disparar trabajo.

MÓDULO 4 · DEL PLAN AL GRAFO

No es otro camino: es el paso siguiente a AIDLC

Inception
Requisitos, historias y unidades de trabajo.

→

Construction
Diseño funcional, NFR e infraestructura de cada unidad.

→

Ola de planificación
Una unidad, o parte de ella, se planifica junta.

→

Tareas
Con dependencias, aceptación y Definition of Ready.

AIDLC define qué y cómo. El plan define en qué orden y cuándo está terminado.

MÓDULO 4 · SKILL CREATE-IMPLEMENTATION-PLAN

De la spec al paquete de tareas

Entra

Spec con nota A o B, design.md, AGENTS.md y el estado del repo.

→

create-implementation-plan

Descompone el alcance en tareas con milestones, dependencias y aceptación.

→

Sale

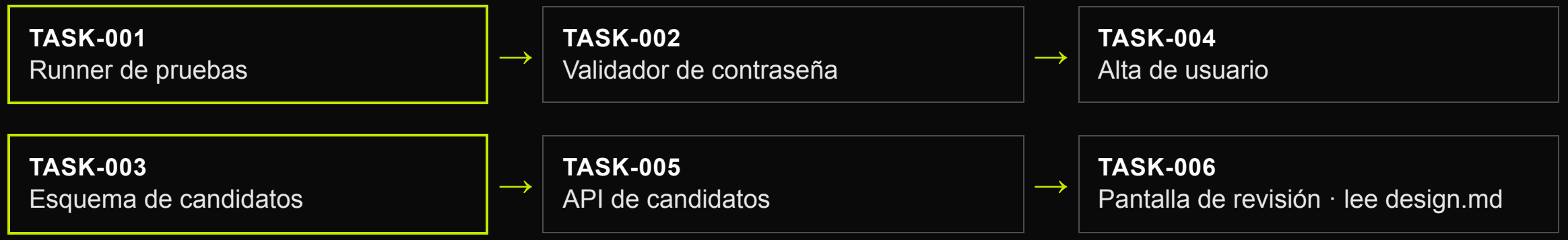
task-package.yaml, milestones.md, un .md por tarea y cobertura.md.

Cada tarea trae alcance, fuera de alcance, entregables, criterios, plan de pruebas y Definition of Ready.

El manifest es la fuente de verdad del paquete. El nombre del archivo no.

MÓDULO 4 · DEPENDENCIAS

El grafo decide qué se puede empezar



☐ Lista: sin bloqueos pendientes ☐ Bloqueada por otra tarea

Un paquete completo puede tener tareas bloqueadas. Sin ciclos, siempre hay una por donde empezar.

MÓDULO 4 · SKILL CONVERT-TASKS-TO-LINEAR

Del paquete a Linear, con red

1 · Validar
IDs únicos, archivos existentes, milestones y referencias válidas.

→

2 · Dry-run
Ver qué se va a crear antes de crearlo.

→

3 · Publicar
Issues, milestones, blockedBy y etiquetas area:.

→

4 · Verificar
Jerarquía, bloqueos y proyecto correcto en Linear.

Sin acceso a Linear: entrega el paquete validado y registra «publicación pendiente».

TU TURNO · 22 MIN

Genera tu grafo

11 min	Ejecuta create-implementation-plan sobre tu spec y design.md.
06 min	En pareja: ¿cada requisito tiene tareas? ¿Hay ciclos? ¿Cuáles están listas?
05 min	Dry-run con convert-tasks-to-linear o carga manual de las primeras tareas.

El paquete completo se termina como preparación de E7. Hoy alcanza con empezarlo bien.

MÓDULO 5 · PUENTE A E7

Una tarea atraviesa el loop

Issue listo
Sin bloqueos, con aceptación.

→

Agente
Lee contexto, implementa, prueba.

→

PR
Diff, tests y evidencia.

→

Review
Humana y automática.

→

Merge
Lo autoriza una persona.

En E7 lo hacemos primero a mano con una tarea. Después el orquestador lo repite para todo el grafo.

Automatiza un ciclo que ya entiendes, no uno que todavía no probaste.

MÓDULO 5 · ORQUESTAR ARNESES

Meta-arnés: un arnés que dirige otros

Arnés coordinador

Claude Code, por ejemplo
planifica, reparte y revisa

CLI headless · ACP

Codex

headless
modelo y plan de OpenAI

OpenHands

arnés abierto
cualquier modelo vía API

Agente ACP

otro proveedor
su modelo y su suscripción

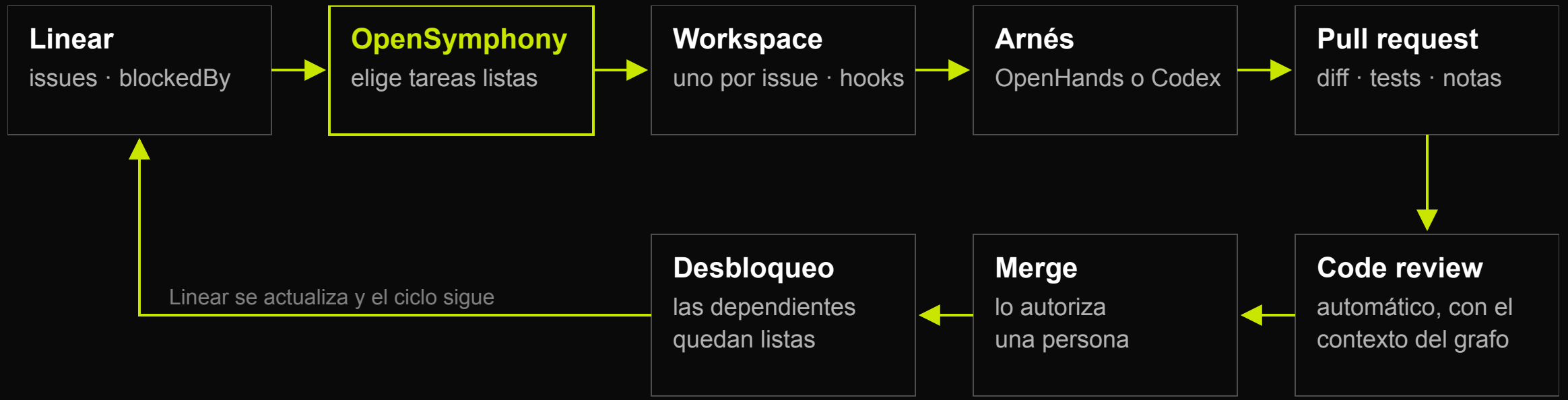
Repo y tareas

Cada trabajador usa su rama y su workspace. La evidencia vuelve al coordinador.

Ejemplos: Orca, Paseo

MÓDULO 5 · ORQUESTACIÓN DE TRABAJO · UN PROYECTO DE LEONARDO GONZÁLEZ

OpenSymphony: el grafo de Linear, ejecutado



Tablero del operador y memoria

Grafo de tareas con ruta crítica, detalle de cada ejecución y un grafo de conocimiento construido con el trabajo terminado.

PREPARACIÓN E7

Antes del lunes: tu rebanada lista

01 Elige una rebanada: un solo flujo que un usuario completa de punta a punta.

02 Escribe la spec mínima: problema, usuario, flujo, 3 criterios y qué queda fuera.

03 Evalúala con evaluate-spec hasta nota A o B.

04 Entorno listo: repo en GitHub, Claude Code en el repo y acceso a Linear.

05 Genera el plan: de 3 a 6 tareas con create-implementation-plan.

Guía completa: estacion7/preparacion.md

RECURSOS · ESTACIÓN 6

Skills, plantillas y lecturas

En este repo

skills/evaluate-spec
skills/create-implementation-plan
plantillas/design.md
ejemplo-y-prompts.md

Afuera

convert-tasks-to-linear · linear
OpenSymphony
Harness engineering · OpenAI
From Spec-Driven Work to Work Orchestration

Lista completa y lecturas opcionales: estacion6/recursos.md